



HADES II

Wikipedia

Xun Felipe Yang

PRW 2º DAW - IES El Rincón

Introduction.....	1
Project Goals.....	2
Project Design.....	3
1. Overall Structure.....	3
2. Database Design.....	4
3. User Interface Design.....	4
4. General Setup.....	8
Remix.....	8
PostgreSQL.....	9
ORM (Prisma).....	10
Tailwind v4.....	14
Shadcn.....	16
Motion.....	18
bcryptjs.....	19
Scripts.....	19
5. Project Development.....	20
Features Recap.....	20
Tailwind v4.....	20
Home Page.....	22
Templates.....	22
User Authentication.....	25
Articles.....	30
6. Optimisations.....	33
Adding FontAwesome.....	33
Minor Adjustments.....	33
Testing and Deployment.....	34
Conclusions.....	37
<i>References.....</i>	<i>38</i>

Introduction

This project consists of developing a wikipedia website for the videogame “Hades 2”.

I have always been a fan of these types of games, and my first ever project was made for the prequel at the time long before the sequel even came out: <https://philipyang89.github.io/hades>

Fast forward a year later and now I have to showcase what I have truly learned and mastered during this 2 year degree.

This time around I will implement some back-end, database and framework knowledge, as my previous project was purely based on raw HTML and CSS. Now I will prove that I have learned some more modern technologies and showcase their use here.

I will be working on this project for roughly 2 hours daily starting late April for around a month. It is a bit of a challenge while doing my Internship at Wellnest GmbH but we will manage.

My personal goal by the end of this is to have successfully built a very solid website that could potentially be commercialized and build a good GitHub freelancer portfolio.

Being able to take on small to mid projects while earning some remuneration would be the best environment to learn and be productive at the same time in my free time.

All of this maintaining a full-time job if possible.

Maybe a bit ambitious? Definitely. But this would be my dream and this is a step towards it.

This document will start on a surface level but will progressively get more technical.

Project Goals

Developing a wikipedia website that would be fast regardless of device or browser, making an emphasis on good performance and loading assets.

Fetching data from an external website to display information such as patch notes history and storing user information privileges inside a database.

Allowing users to like or comment on the wiki entries - Admins would be able to delete entries but Users would not be able to.

Everyone should also be allowed to edit their own comments.

Making the website accessible according to WCAG (Web Content Accessibility Guidelines) and responsive to all devices while guaranteeing interface and user experience flow is a must.

A beautiful landing page on arrival would be a pretty nice feature to have.

Color palette and asset usage will be as loyal as possible to the game design.

Learning about more complex animations in-depth using CSS and JavaScript.

Using popular and modern frameworks in order to adapt to the current market trends in the sector of Web Development.

Optimising code length as far as possible so that it is more readable and maintainable.

Most importantly, learn :)

Project Design

There really are endless frameworks nowadays, it is a very hot and controversial topic and many times people make their own decisions based on what they want to achieve or whether one is more popular than the other and so on...

In my case I wanted to try a full-stack framework, as they seem to be on the rise for what is supposed to make your life a bit easier. The more popular react based options now are Next.js and Remix.js without a doubt. Laravel or Symfony were next in the list as strict MVC frameworks but having already worked with Laravel I opted for the aforementioned react options.

Why Remix instead of Next?

That is a good question, and the answer in my case would be the affiliation Remix has with Vercel that is pushing the framework in the direction of becoming a product more than being an open-source project.

Remix has the downside of being a newer framework and perhaps lacking a bigger community but their emphasis on web fundamentals and simplicity definitely catches my eye.

1. Overall Structure

The Tech Stack I am planning to use is the following:

Frontend & Backend:

- Remix.js which is a full-stack framework built on top of React.
- shadcn/ui provides some prebuilt UI components whose styling is based on Tailwind v4.
- Radix UI as component logic - since shadcn is built on top of this.
- Motion as an animation library with a wide support in React (Javascript and Vue too)

Database & ORM:

- PostgreSQL as our relational database, it is heavier than SQLite but it's worth trying.
- Prisma as our Object Relational Mapper for better interaction between the DB.

APIs:

- Steam News API (?) to fetch Patch Notes history from an official source.

Deployment (Long-lived server):

- Render: git based deployments, has high bundle limits and supports PostgreSQL.

2. Database Design

As for how the tables would be structured, the following entities would be required:

- enum Role, model User, Article, Comment and Like.

The User would have a Role and Comments + Likes made.

An Article would have Comments associated but no User relation since we're manually creating.

Comments would have an User and Article dependency + Likes count.

Likes would finally have a Comment and User linked to it.

3. User Interface Design

To accomplish the most similar design to the game possible - I will extract all possible assets from illustrations, fonts and even color palettes available.

Here is the public list of files of the latest update: <https://steamdb.info/depot/1145352/>

The files needed are compressed in packages to limit their use (.xnb/.pkg extensions) as Hades 2 uses LZ4 compression. In order to gather these, I involve myself in the modding community of the game and find out this open-source project programmed with Python to do it efficiently.

Name is [depPTH](#) and it is maintained by a guy named quærus. The dependencies needed would be LZ4 logically - and Pillow, which is an Imaging Library on Python, that provides the necessary image processing capabilities to export the images.

```
pip install lz4
pip install pillow
```

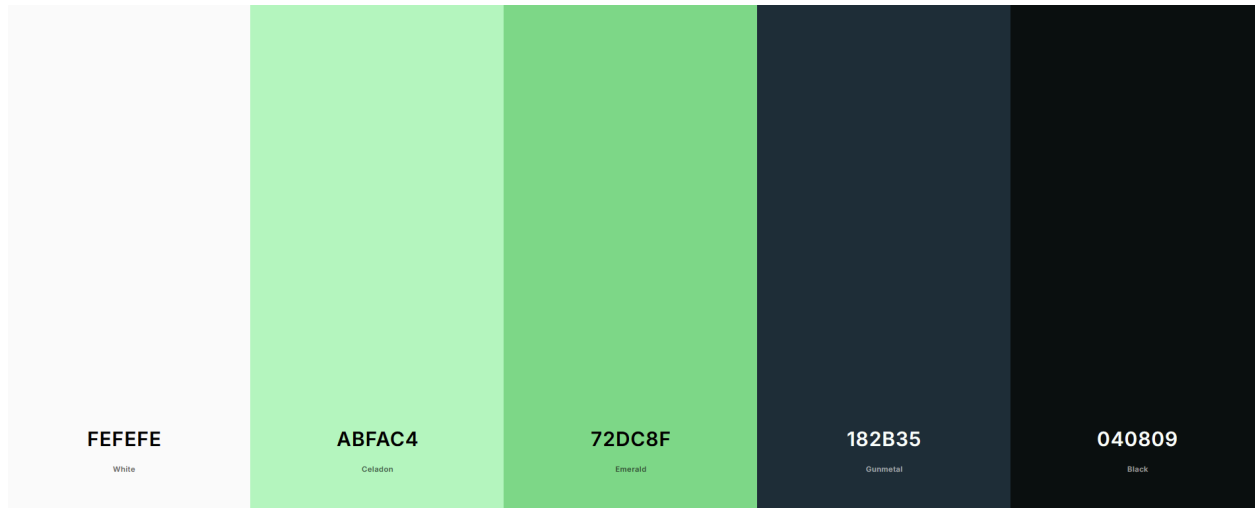
After successfully setting up the tool, we will run this script inside our desired folder destination:

```
$packages = Get-ChildItem *.pkg -Recurse
foreach ($package in $packages) {
    Depth ex $package -s
```

```
}
```

Once exported all the Assets, I will proceed to make a mockup of how would some pages look like making use of Figma and a color palette I decided on:

Color Palette: <https://coolors.co/fefefe-abfac4-72dc8f-182b35-040809>



White: **#FEFEFE** , Light Green: **#ABFAC4** , Green: **#72DC8F** , Light Black: **#182B35**, Black: **#040809**

Using FontAwesome v6 for the icons in the website: <https://fontawesome.com/v6/search>

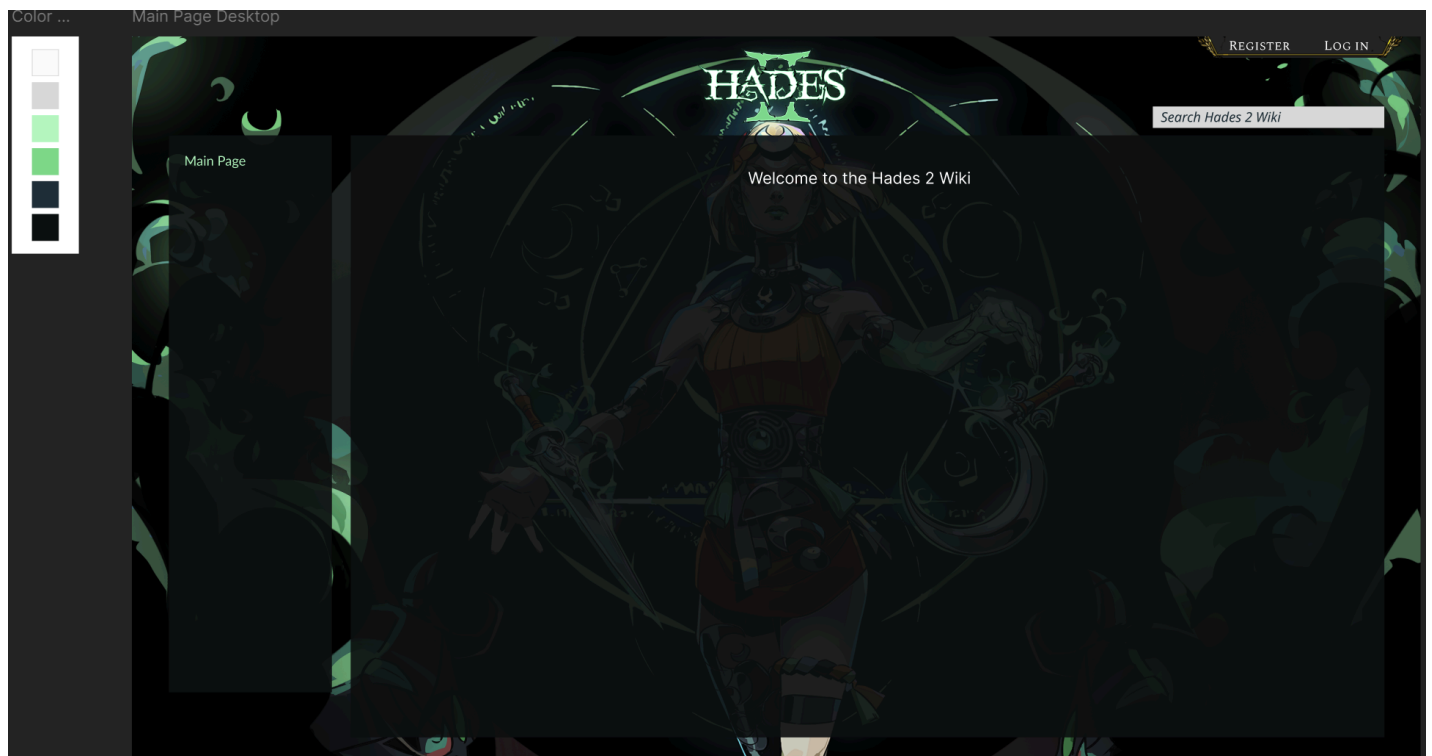
These are the Fonts used in-game which I will use for the project:

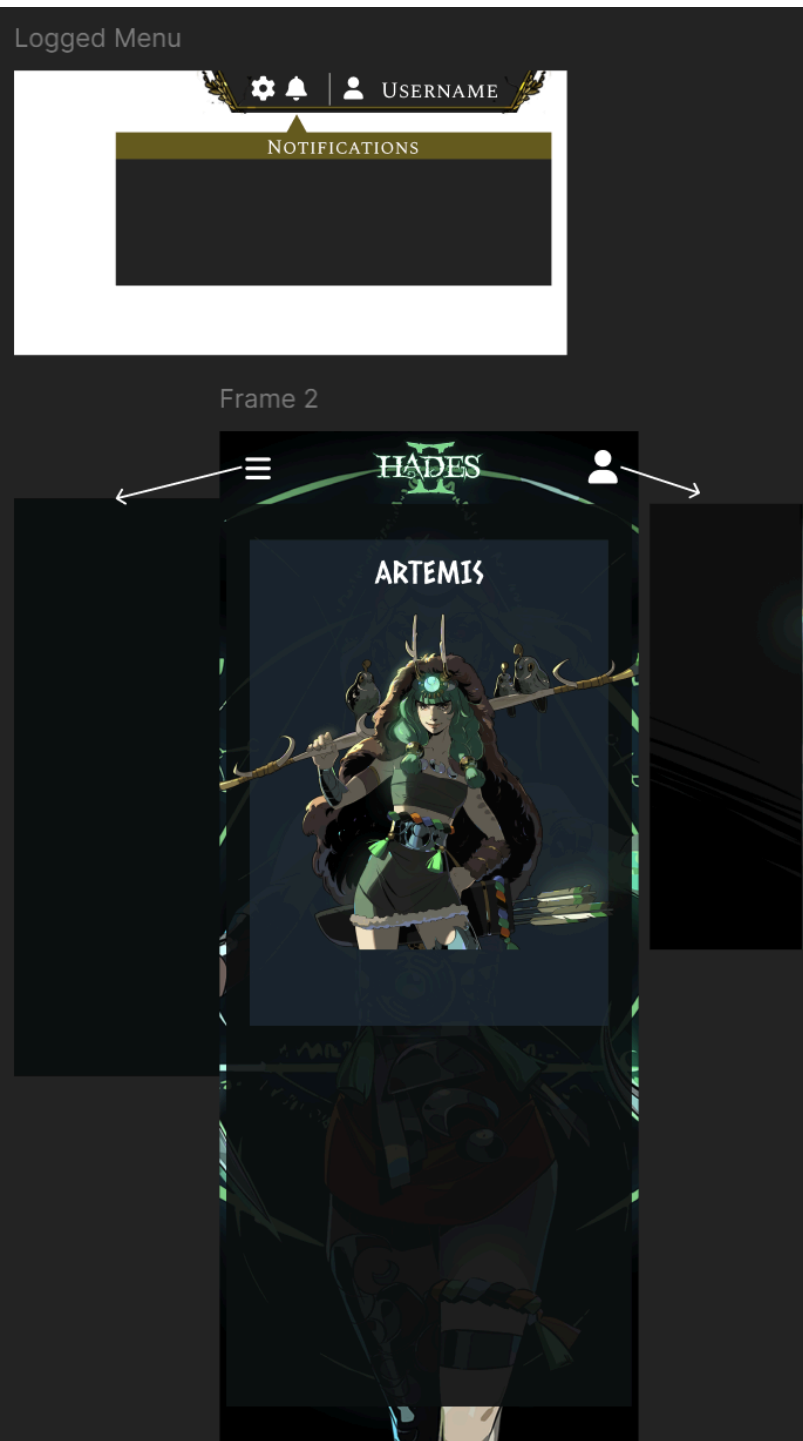
- Noto Sans (Text)
- Lato (Index, Menu)
- Spectral SC (Titles, Headers)
- Caesar Dressing (Character Names)
- Crimson Text Italic (Quotes)

Link to Google Fonts:

<https://fonts.google.com/share?selection.family=Caesar+Dressing|Crimson+Text:ital@1|Lato:ital,wght@0,300;0,400;0,700;0,900;1,300;1,400;1,700;1,900|Noto+Sans|Spectral+SC:wght@200;300;400;500;600;700>

[Figma Design Mockup Link](#)





Mobile view

4. General Setup

Remix

To start setting our environment app, we will have to pull the latest remix version. I will try to install everything in order so it is easier to grasp.

I would also love to debate which packet manager is better, but to make things simple I will just stick to using pnpm as it is faster, saves up disk space and is the more modern option.

If you work in a larger team and are also using some older packages you should stick to npm.

- Installing pnpm is as easy as just running this npm command: `npm install -g pnpm`

```
C:\Users\Philip>pnpm -v
10.11.0
```

```
pnpm npm yarn bun
```

```
pnpm create remix@latest my-app
```

```
PS C:\Users\Philip\Documents> pnpm create remix@latest hades2-wiki
WARN 2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
.../1972377d9b5-23e8 | +205 ++++++
.../1972377d9b5-23e8 | Progress: resolved 205, reused 0, downloaded 205,

remix v2.16.8 Let's build a better website...
  Directory: Using hades2-wiki as project directory
  Using basic template See https://remix.run/guides/templates for more
  ✓ Template copied

git Initialize a new git repository?
Yes

deps Install dependencies with pnpm?
Yes

  ✓ Dependencies installed
  ✓ Git initialized

done That's it!

Enter your project directory using cd .\hades2-wiki
Check out README.md for development and deploy instructions.

Join the community at https://rmx.as/discord
```

Since we initialized the git repository, we only have to add the following:

```
git remote add origin https://github.com/philipyang89/hades2-wiki
```

In order to work more efficiently, a good version control setup is needed.

The following command is needed since pnpm generates .yaml files and git warns us often:

```
git config --global core.autocrlf true
```

I am going to create an initial latest untouched Remix main branch so that I can create different working branches from there and traceback any future errors.

```
git branch -M main  
git push -u origin main  
git checkout -b dev
```

Now that we are working on the dev branch, we can do “git add” and “git commit -m” normally.

PostgreSQL

After having PostgreSQL installed with pgAdmin 4, we will use that to locally check our DB.

We should also add our bin path as an Environment Variable so we can run commands in cmd.

C:\Program Files\PostgreSQL\17\bin

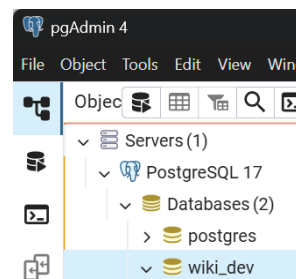
```
PS C:\Users\Philip\Documents\dev\hades2-wiki> createdb --version  
createdb (PostgreSQL) 17.5
```

This means that we successfully added the path and integrated its CLI commands.

To create a new database it is important to do it with our “postgres” root user, there are ways of working with your current User but it requires some setup.

```
createdb -U postgres wiki_dev
```

We could then check inside the DB inside pgAdmin 4 →



ORM (Prisma)

To correctly setup Prisma now, we will need the CLI dev and the runtime dependency.

```
pnpm add -D prisma
pnpm add @prisma/client
```

```
PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm add -D prisma
WARN deprecated eslint@8.57.1: This version is no longer supported. Please see https://eslint.org/version-support for other options.
Downloading prisma@6.8.2: 6.21 MB/6.21 MB, done
WARN 5 deprecated subdependencies found: @humanwhocodes/config-array@0.13.0, @humanwhocodes/object-schema@2.0.3, glob@7.2.3, inflight@1.0.6, rimraf@3.0.2
Packages: +12
+++++++
Progress: resolved 893, reused 782, downloaded 8, added 12, done

devDependencies:
+ prisma 6.8.2

Warning
Ignored build scripts: @prisma/engines, prisma.
Run "pnpm approve-builds" to pick which dependencies should be allowed to run scripts.
```

Deprecations are totally normal but what we should do is to run “pnpm approve-builds” after.

```
PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm approve-builds
√ Choose which packages to build (Press <space> to select, <a> to toggle all, <i> to invert selection)
All packages were added to ignoredBuiltDependencies.
```

We can safely start installing Prisma now.

```
pnpm exec prisma init
```

```
PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm exec prisma init
Fetching latest updates for this subcommand...

✓ Your Prisma schema was created at prisma/schema.prisma
You can now open it in your favorite editor.

warn You already have a .gitignore file. Don't forget to add '.env' in it to not commit any private information.

Next steps:
1. Set the DATABASE_URL in the .env file to point to your existing database. If your database has no tables yet, read https://pris.ly/d/getting-started
2. Set the provider of the datasource block in schema.prisma to match your database: postgresql, mysql, sqlite, sqlserver, mongodb or cockroachdb.
3. Run prisma db pull to turn your database schema into a Prisma schema.
4. Run prisma generate to generate the Prisma Client. You can then start querying your database.
5. Tip: Explore how you can extend the ORM with scalable connection pooling, global caching, and real-time database events. Read: https://pris.ly/cli/beyond-orm

More information in our documentation:
https://pris.ly/d/getting-started
```

It is important to now change the .env Database_URL to the “postgres” root account and set the password in the highlighted part, followed by the 5432 default port and the DB name at the end.

```
DATABASE_URL="postgresql://postgres:123@localhost:5432/wiki_dev"
```

The provisional DB schema we define inside Prisma would be the following:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  USER
  ADMIN
}

model User {
  id        String   @id @default(cuid())
  email     String   @unique
  password  String
  role      Role     @default(USER)
  comments  Comment[]
  likes     Like[]
}

model Article {
  id        String   @id @default(cuid())
  title     String
  slug      String   @unique
  content   String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  comments  Comment[]
}
```



```

}

model Comment {
  id          String      @id @default(cuid())
  content     String
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  article     Article     @relation(fields: [articleId], references:
[id])
  articleId   String
  user        User        @relation(fields: [userId], references: [id])
  userId      String
  parent      Comment?    @relation("CommentReplies", fields: [parentId],
references: [id])
  parentId   String?      // For replies
  replies     Comment[]   @relation("CommentReplies")
  likes       Like[]
}

model Like {
  id          String      @id @default(cuid())
  user        User        @relation(fields: [userId], references: [id])
  userId      String
  comment      Comment     @relation(fields: [commentId], references:
[id])
  commentId   String
  createdAt   DateTime    @default(now())
  notified    Boolean      @default(false) // To mark if notification sent

  @@unique([userId, commentId]) // Only one like per user per comment
}

```

To now apply the schemas we will do the following:

```

pnpm exec prisma migrate dev --name init
pnpm exec prisma generate

```

```

PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm exec prisma migrate dev --name init
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma
Datasource "db": PostgreSQL database "wiki_dev", schema "public" at "localhost:5432"

Applying migration `20250531233434_init`

The following migration(s) have been created and applied from new schema changes:

migrations/
├─ 20250531233434_init/
│   └─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (v6.8.2) to .\node_modules\.pnpm\@prisma+client@6.8.2_prisma_8

```

```

PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm exec prisma generate
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma

✓ Generated Prisma Client (v6.8.2) to .\node_modules\.pnpm\@prisma+client@6.8.2_

Start by importing your Prisma Client (See: https://pris.ly/d/importing-client)

Tip: Want to turn off tips and other hints? https://pris.ly/tip-4-nohints

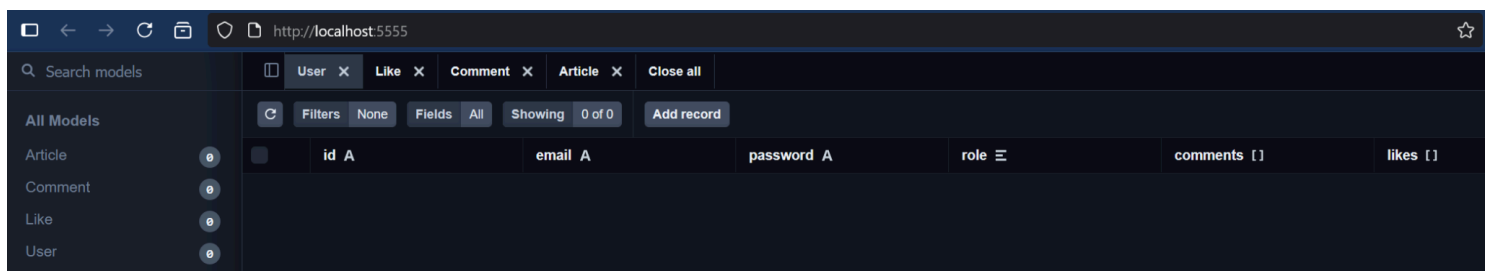
```

If we now want to access the Database and check rows visually we can run Prisma Studio:

```

PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm exec prisma studio
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma
Prisma Studio is up on http://localhost:5555

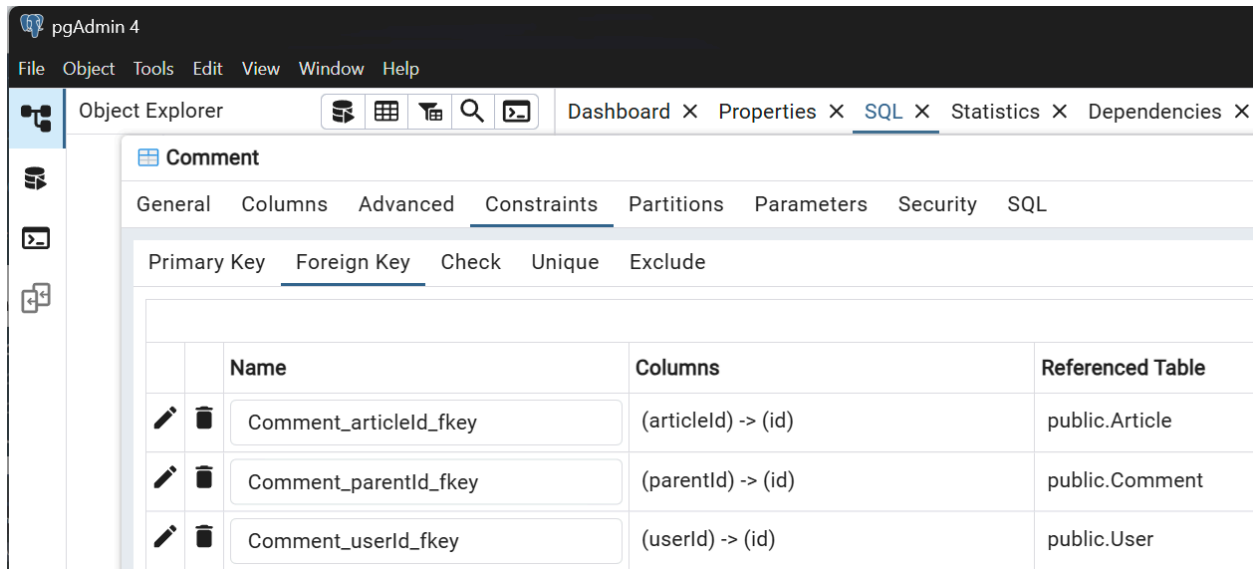
```



You could also access the Database from pgAdmin 4 but it is a bit more complex:

- Servers → Databases → (DB Name) → Schemas → public → Tables

It is more useful in case you want to check the Constraints we defined in the schema before:



Or just need more in-depth information regarding your Database in general.

I will mainly be using my Prisma Studio in the future.

Tailwind v4

It is important we update our stack to Tailwind v4 before, as shadcn does require v4 to fully have access to their tools and components. You could theoretically install shadcn before and then update Tailwind but I think it's probably more complicated to do so.

Before anything, I will commit my changes before upgrading.

```
pnpm add -D tailwindcss@latest
npx @tailwindcss/upgrade --force
```



Now that the upgrade is done, the most important thing is to change what our `tailwind.css` was before, we can remove all `base/components/utilities` and use a single import line.

Next would be removing unneeded dependencies.

```
Additionally, in v4 imports and vendor prefixing is now handled for you automatically, so you can remove 'postcss-import' and 'autoprefixer' if they are in your project:
```

```
pnpm remove autoprefixer postcss
```

Since Remix uses Vite for deployments we will also have to edit the `"vite.config.ts"` file in our root folder for it to detect our newest Tailwind version. For that we will install its plugin.

```
pnpm add -D @tailwindcss/vite
```

The migration from PostCSS to Vite in v4 is one of the reasons the performance is much better.

```
vite.config.ts > [v] default > [v] plugins
1  import { vitePlugin as remix } from "@remix-run/dev";
2  import { defineConfig } from "vite";
3  import tsconfigPaths from "vite-tsconfig-paths";
4  import tailwindcss from "@tailwindcss/vite";
5
6  declare module "@remix-run/node" {
7    interface Future {
8      v3_singleFetch: true;
9    }
10 }
11
12 export default defineConfig({
13   plugins: [
14     tailwindcss(),
15     remix({
```

We edit our `"vite.config.ts"` this way

Additionally, we have to edit our `root.tsx` so that `"tailwind.css"` is read from an URL inside the head content of our HTML.

```

app > root:tsx > Layout
1  import tailwindStyleSheetUrl from "../tailwind.css";
2  import {
3    Links,
4    Meta,
5    Outlet,
6    Scripts,
7    ScrollRestoration,
8  } from "@remix-run/react";
9  import type { LinksFunction } from "@remix-run/node";
10
11 export const links: LinksFunction = () => [
12   { rel: "stylesheet", href: tailwindStyleSheetUrl },
13   { rel: "preconnect", href: "https://fonts.googleapis.com" },
14   {

```

Shadcn

We now should be able to run the CLI init command:

```
pnpm dlx shadcn@latest init
```

```

● PS C:\Users\Philip\Documents\dev\hades2-wiki> pnpm dlx shadcn@latest init
✓ Preflight checks.
✓ Verifying framework. Found Remix.
✓ Validating Tailwind CSS config. Found v4.
✓ Validating import alias.
✓ Which color would you like to use as the base color? » Slate
✓ Writing components.json.
✓ Checking registry.
✓ Updating CSS variables in app\tailwind.css
✓ Installing dependencies.
✓ Created 1 file:
  - app\lib\utils.ts

Success! Project initialization completed.
You may now add components.

```

Shadcn configuration is stored inside components.json*

Getting the Tailwind v4 configuration right for shadcn took a bit more than I expected but it will hopefully be worth the setup, as it introduces many interesting cool features.

The only downside is that some older browsers might not be fully compatible.

Having that said, pulling shadcn added a ton of lines to the tailwind.css and we can instantly notice that all variables are now using the newer “oklch” colour standard that is better than rgb or hsl and preferred for accessibility, gradients, dark modes and more advanced theming.

<https://evilmartians.com/chronicles/oklch-in-css-why-quit-rgb-hsl> ← This is an interesting read.



So, in order to add my Custom Variables now, I will proceed to convert rgb into oklch and then create a new globals.css file but import it at the end of the tailwind.css file.

```
130 | @import "./globals.css";
```

The reason is very simple, many other frameworks expect a single CSS file and despite Remix allowing multiple css files inside its “root.tsx” we will do it this way since some pipelines also do not allow for multiple CSS files.

We add the import line at the end so that globals.css > tailwind.css in terms of variable priority.

```

app > globals.css > :root
1  :root {
2      --white: oklch(0.997 0 0);
3      --grey: oklch(0.8853 0 0);
4      --light-green: oklch(0.9197 0.1072 153.97);
5      --green: oklch(0.8115 0.1472 150.83);
6      --light-black: oklch(0.2775 0.0309 232.41);
7      --black: oklch(0.129 0.0093 211.36);
8  }

```

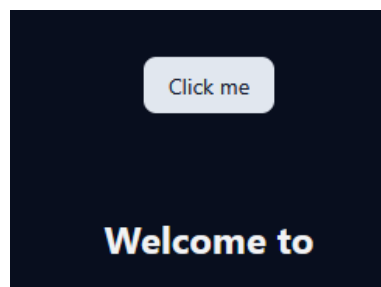
My small globals.css file for now

In order to try any shadcn component we only have to import the component we want inside our “components\ui” path, and then just import it into our .tsx file and call it.

```

app > routes > _index.tsx > Index
1  import type { MetaFunction } from "@remix-run/node";
2  import { Button } from "~/components/ui/button";
3
4  export const meta: MetaFunction = () => {
5      return [
6          { title: "New Remix App" },
7          { name: "description", content: "Welcome to Remix!" },
8      ];
9  };
10
11 export default function Index() {
12     return (
13         <div className="flex h-screen items-center justify-center">
14             <div className="flex flex-col items-center gap-16">
15                 <Button>Click me</Button>
16             </div>
17         </div>
18     );
19 }

```



Motion

Formerly known as Framer Motion, is a pretty good animation library I want to add and probably give some use to familiarise myself with this tool as many new websites use it actively.


```
pnpm install motion # 12.15.0
```

So now we can just import motion into any component with a simple:

```
import { motion } from "motion/react"
```

I think I will showcase the use of Motion later on in the documentation more in-depth and see if we can get good animation results easily or not so easily.

bcryptjs

As an extra layer of security for Authentication, it acts as a Password hasher and verifier.

```
pnpm add bcryptjs # 3.0.2
```

We could add a package called “remix-auth” to simplify logins but I will avoid it in order to fully customize our own Sign-in flows and get a grasp of how it works from the lowest level.

Scripts

To run commands faster and later to deploy our project on either development or production, I will write down some useful commands that will later be written on the README file too.

Our scripts section inside our package.json will then look like this:

```
"scripts": {  
  "build": "remix vite:build",  
  "dev": "remix vite:dev",  
  "prisma:migrate": "prisma migrate dev --name init && prisma generate",  
  "tailwind:upgrade": "npx @tailwindcss/upgrade --force",  
  "lint": "eslint --ignore-path .gitignore --cache --cache-location  
./node_modules/.cache/eslint .",  
  "start": "remix-serve ./build/server/index.js",  
  "typecheck": "tsc"  
},
```

In dev: “pnpm dev” In prod: “pnpm build” , “pnpm start”

5. Project Development

Features Recap

I am going to make a schema of the main features I want to have in the project for organisation purposes and to make a sort of checklist.

Hero:

- Animated mp4 video background, plays on loop with some elements.
- Click to redirect into the main page or landing.

Articles:

- Created/seeded locally without any app editing, for now.
- View by URL slug, automatically added into the sidebar.
- Templates made for responsiveness.

Auth:

- Login/Register redirecting into separate pages.
- Session-based login with cookies without JWTs.
- Password hashing with bcryptjs.
- Login can be made with either Email or Username

Tailwind v4

So, to start off we are going to import our Google Fonts inside our “root.tsx” export of Remix

```
export const links: LinksFunction = () => [  
  { rel: "stylesheet", href: tailwindStylesheetUrl },  
  { rel: "preconnect", href: "https://fonts.googleapis.com" },  
  {  
    rel: "preconnect",  
    href: "https://fonts.gstatic.com",  
    crossOrigin: "anonymous",  
  },  
  {  
    rel: "stylesheet",  
    href: "https://fonts.googleapis.com/css2?family=Caesar+Dres",  
  },  
];
```

We will then add them inside our theme section in our tailwind.css, font sizes and bold or italic can be handled by Tailwind with simple inline classes. (Usually could be made in config.ts)

```
12  .font-lato {
13    font-family: "Lato", sans-serif;
14  }
15  .font-noto {
16    font-family: "Noto Sans", sans-serif;
17  }
18  .font-spectralsc {
19    font-family: "Spectral SC", serif;
20  }
21  .font-caesar {
22    font-family: "Caesar Dressing", system-ui;
23  }
24  .crimson-text {
25    font-family: "Crimson Text";
26  }
```

Using “arbitrary values” we can then call upon global variables I have in globals.css directly.

```
<Link
  to="/main"
  className="redirect-main font-spectralsc text-4xl font-medium pb-8 transition"
>
  To Main Page
</Link>
```

```
.redirect-main {
  padding-bottom: 10rem;
  display: inline-block;
  transition: color 0.25s;
  color: var(--green);
}
.redirect-main:hover {
  color: var(--light-green);
}
```



My cheatsheet for styling in Tailwind v4 is probably going to be something like:

<https://tailwindcss.com/docs/font-size>

<https://tailwindcss.com/docs/font-weight>

<https://tailwindcss.com/docs/padding>

<https://tailwindcss.com/docs/colors>

Basically anything you can find in the documentation, and after some time I started noticing how convenient it can be over traditional CSS class defining hell.

Home Page

First of all, we define a simple background video playing on loop and stored locally.

We then add a <Link> element to redirect the User into the main page.



Templates

Before going any further I would like to define a general template for all the wikipedia pages or entries, however you want to call it.

Modularisation here is very important so I will create different components and then call them.

```
app/  
  components/  
    ui/  
      pagetemplate.tsx  
      sidebararticles.tsx  
      mobiletopbar.tsx  
      article.tsx  
      ... (shadcn components)  
  routes/
```

```
_index.tsx
main.tsx
register.tsx
login.tsx
article.$slug.tsx
... (more articles)
list.ts
root.tsx
tailwind.css
globals.css
```

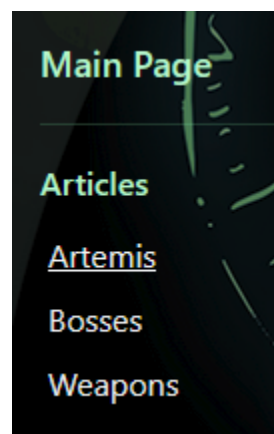
This is the path and folder structure I will be using to modularise all the pages and have a layout that is as consistent as possible.

Article creation will be manual but in the future could be added to the Database to allow for editing or creation by Admin users.

The core of all is the “pagetemplate.tsx”, here we define the behaviour and the responsiveness of our page. We define our blocks so to speak and make it so that under 600px we get a different view to accommodate for mobile users.

We create a “sidebararticles.tsx” because we want to automatically update our Sidebar or Index by manually adding our records in a “list.ts” file and then just adding our route article.\$slug.

```
export const articles = [
  {
    title: "Artemis",
    slug: "artemis", // co
  },
  {
    title: "Bosses",
    slug: "bosses",
  },
  {
    title: "Weapons",
    slug: "weapons",
  },
];
```



Our PageTemplate will make use of our “mobiletopbar.tsx” component to handle responsiveness

We will also add “useState” to detect when our screen reaches the size to change its layout, we start by making the mobile elements appear hidden at first but then change to flex and so on.

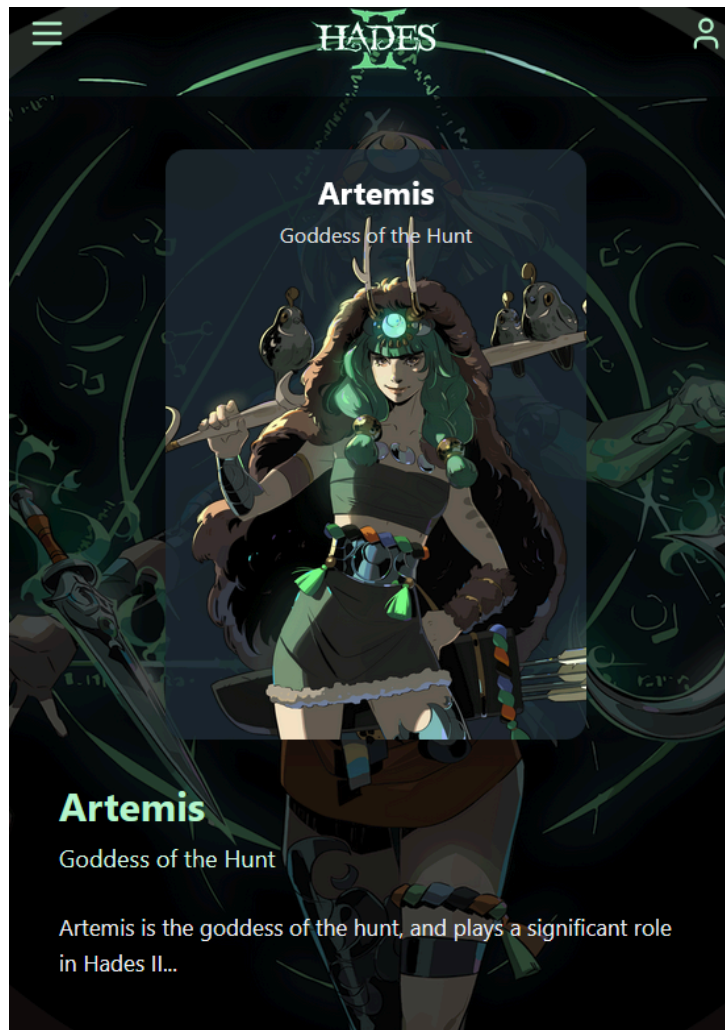
```
app > components > ui > pagetemplate.tsx > PageTemplate
1  import { ReactNode, useState } from "react";
2  import { Link } from "@remix-run/react";
3  import MobileTopBar from "app/components/ui/mobiletopbar";
4
5  type PageTemplateProps = {
6    children: ReactNode;
7    sidebar?: ReactNode;
8  };
9
10 export default function PageTemplate({ children, sidebar }: PageTemplateProps) {
11   const [mobileSidebar, setMobileSidebar] = useState(false);
12   const [mobileAuth, setMobileAuth] = useState(false);
13 }
```

A big mistake I found in between was that Vite doesn't like importing Tailwind.css the way we should do it, so instead we have to just simplify it to:

```
1 import "../tailwind.css";
```

Found this out because all the buttons and scripts were not working at all.





User Authentication

```
2 | import { Link } from "@remix-run/react";
```

We first create the Links to redirect the user into different pages.

```
<Link to="/register">  
  Register  
</Link>  
<Link to="/login" cL  
  Log in  
</Link>
```

To create the User Authentication logic we will need to create 3 new files:

- db.server.ts
- session.server.ts
- auth.server.ts

But before we even start doing anything we have to generate a prisma client like so:

```
npx prisma generate
```

To briefly explain, we are going to implement a Cookie Session instead of the other popular JWT alternative for User Authentication. Reason being that Cookie is managed in-built into Remix.

We are basically setting a cookie on the user's browser whenever it logs in or registers.

That cookie has the user's ID encrypted with a session secret so for future requests we just read that cookie and if the userID is found we load the user from the DB.

It is also very secure as we are using httpOnly

Our standard "db.server.ts" will look like this:

```
import { PrismaClient } from "@prisma/client";

let prisma: PrismaClient;

declare global {
  var prisma: PrismaClient | undefined;
}

if (process.env.NODE_ENV === "production") {
  prisma = new PrismaClient();
} else {
  // Prevents multiple client instances in dev
  if (!global.prisma) {
    global.prisma = new PrismaClient();
  }
  prisma = global.prisma;
}

export default prisma;
```

We also prevent multiple client instances being generated.

The session.server.ts file will look like this:

```
app > session.server.ts > ...
1 import { createCookieSessionStorage, redirect } from "@remix-run/node";
2
3 // Use a Long, unpredictable string for your secret, ideally loaded from ENV
4 const sessionSecret = process.env.SESSION_SECRET || "SUPER_SECRET_SESSION_KEY";
5
6 // Create cookie session storage
7 export const sessionStorage = createCookieSessionStorage({
8   cookie: {
9     name: "__session",
10    secure: process.env.NODE_ENV === "production",
11    secrets: [sessionSecret],
12    sameSite: "lax",
13    path: "/",
14    httpOnly: true,
15    maxAge: 60 * 60 * 24 * 30, // 30 days
16  },
17 });
18
19 // Helpers to work with sessions
20 export async function getSession(request: Request) {
21   return sessionStorage.getSession(request.headers.get("Cookie"));
22 };
23 export async function commitSession(session: Awaited<ReturnType<typeof getSession>>) {
24   return await sessionStorage.commitSession(session);
25 }
26 export async function destroySession(session: Awaited<ReturnType<typeof getSession>>) {
27   return await sessionStorage.destroySession(session);
28 }
29
30 // Utility: require user id (redirects if not present)
31 export async function requireUserId(request: Request, redirectTo: string = "/login") {
32   const session = await getSession(request);
33   const userId = session.get("userId");
34   if (!userId) throw redirect(redirectTo);
35   return userId as string;
36 }
37
38 // Utility: get user id (returns string | undefined)
39 export async function getUserId(request: Request) {
40   const session = await getSession(request);
41   const userId = session.get("userId");
42   return typeof userId === "string" ? userId : undefined;
43 }
```

We first import tools from in-built Remix as I specified above (CookieSession) that will allow us to create server-side sessions stored in HTTP Cookies.

We import redirect in order to later redirect the user to different pages.

Session Secret and Storage are later created and safely encrypted.

We then have Session Helpers that will read this information.

Later down the file we have some logic to require or get UserID.

Before we proceed to define my auth.server.ts I want to redefine our goals and let the Users login with both their Username or Emails, but for that I have to add a new table row.

```
21 model User {
22   id      String  @id @default(cuid())
23   email   String  @unique
24   username String  @unique
25   password String
26   role    Role    @default(USER)
27   comments Comment[]
28   likes   Like[]
29 }
```

In order to add an additional field to the User database we will have to delete our migrations (in case we don't mind getting rid of the data) and then run: `pnpm prisma migrate reset`

We can then safely do `pnpm prisma:migrate` to run the 2 commands we have on our package.json.

```
"prisma:migrate": "prisma migrate dev --name init && prisma generate",
```

Comment

×

Like

×

Article

×

User

×

Close all

Filters

None

Fields

All

Showing

1 of 1

Add record

	id A	email A	username A?	password A	role
	cmbi5z2x10000x748ynjdh9...	test@gmail.com	Test	\$2b\$10\$2Je/v4KM8UH6ANr...	USER

This is how one entry would be structured from prisma studio

Inside our /register route:

```
const username = formData.get("username");
if (typeof username !== "string" || !username.trim()) {
  return json({ error: "Username required" }, { status: 400 });
}
try {
  // register returns a redirect response on success
  return await register({ email, password, username });
} catch (error: any) {
  return json(
    { error: error.message || "Registration failed" },
    { status: 400 }
  );
};
```

We add an input in our Form:

```
<input
  name="username"
  type="text"
  placeholder="Username"
  required
  className="input"
  autoComplete="username"
/>
```

We finally have to edit our auth.server.ts which is what takes care of the backend to allow both username or emails for login, we have to create an “identifier” class that could use either:

```
export async function login({ identifier, password }: { identifier: string, password: string }) {
  const user = await prisma.user.findFirst({
    where: {
      OR: [
        { email: identifier },
        { username: identifier }
      ]
    }
  });
  if (!user) throw new Error("Invalid email/username or password");
  const isValid = await compare(password, user.password);
  if (!isValid) throw new Error("Invalid email/username or password");
  return await createUserSession(user.id, "/");
}
```

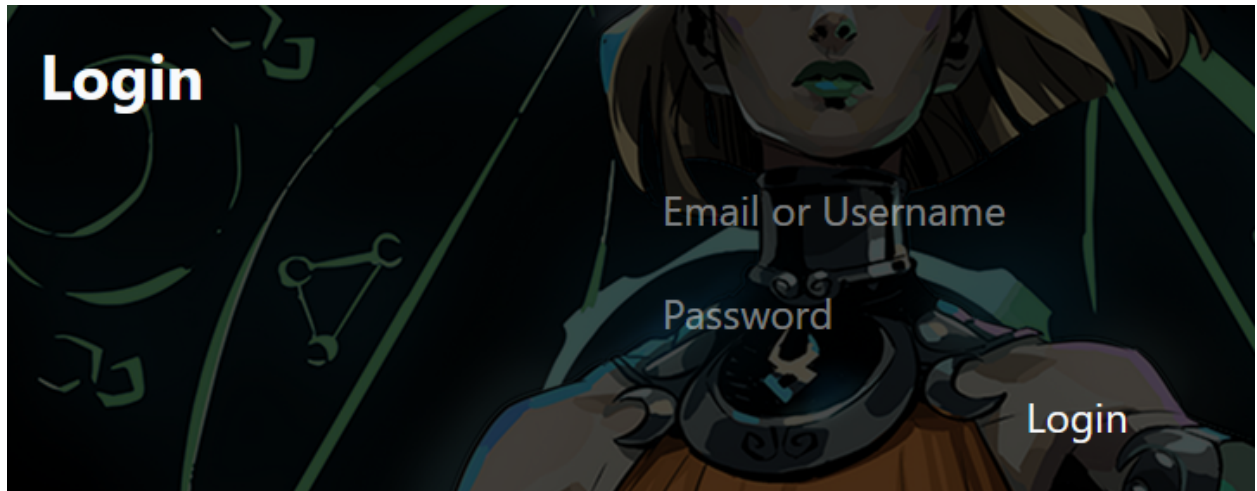
Then in our login.tsx we change what we await from “email” to “identifier”:

```
export async function action({ request }: ActionFunctionArgs) {
  const formData = await request.formData();
  const email = formData.get("email");
  const password = formData.get("password");
  if (typeof email !== "string" || typeof password !== "string") {
    return json({ error: "Invalid form submission" }, { status: 400 });
  }
  try {
    // Login returns a redirect response on success
    return await login({ email, password });
  } catch (error: any) {
    return json({ error: error.message || "Login failed" }, { status: 400 });
  }
}
```

```
export async function action({ request }: ActionFunctionArgs) {
  const formData = await request.formData();
  const identifier = formData.get("identifier");
  const password = formData.get("password");
  if (typeof identifier !== "string" || typeof password !== "string") {
    return json({ error: "Invalid form submission" }, { status: 400 });
  }
  try {
    // Login returns a redirect response on success
    return await login({ identifier, password });
  } catch (error: any) {
    return json({ error: error.message || "Login failed" }, { status: 400 });
  }
}
```

Finally, we can just edit
the form to have a placeholder
say to input either email or
username for the User. Max Length = 10

```
<input
  name="identifier"
  type="text"
  placeholder="Email or Username"
  required
  className="input"
  autoComplete="username"
  maxLength={10}
/>
```



This is how our Login route would now look like

Articles

Finally, I am going to link the Articles to our Database, as I was manually creating them inside our local environment. But it is ideal to do this in order to possibly add Nested Comments in the future and just save Articles in a safer way. This is called Dynamic DB-Powered Articles.

We will start by changing the Article Table to allow storing wiki values like subtitles and image routes in our database.

```
31  model Article {
32    id      String    @id @default(cuid())
33    title   String
34    slug    String    @unique
35    subtitle String?
36    image   String
37    imageAlt String?
38    content String
39    createdAt DateTime @default(now())
40    updatedAt DateTime @updatedAt
41    comments Comment[]
42  }
```

“prisma migrate dev” after

We follow by replacing our Single Source of Truth from list.ts to basically a seeding script that I will call seed.js inside my /prisma folder.

```

import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

const articles = [
  {
    title: "Artemis",
    slug: "artemis",
    subtitle: "Goddess of the Hunt",
    image: "/images/Artemis.png",
    imageAlt: "Statue of Artemis",
    content: `
<p>Artemis is the goddess of the hunt, and plays a significant role in Hades
II...</p>
`,
  },
  // Add more articles...
];

async function main() {
  for (const article of articles) {
    await prisma.article.upsert({
      where: { slug: article.slug },
      update: {},
      create: article,
    });
  }
}

main().then(() => prisma.$disconnect());

```

In order to be able to run the seed we first define this inside our package.json:

```

"prisma": { "seed": "node prisma/seed.js" },

```

We then run the seed to update our DB:

```

npx prisma db seed

```

Comment	Article	User	Like	Close all
---------	---------	------	------	-----------

Filters	None	Fields	All	Showing	1 of 1	Add record
---------	------	--------	-----	---------	--------	------------

id A	title A	slug A	subtitle A?	image A	imageAlt A?	content A
cmbifrng10000qb0cupdjua...	Artemis	artemis	Goddess of the Hunt	/images/Artemis.png	Statue of Artemis	<p>Artemis is the godde...

We can then verify that the seed is working correctly and we can now add/edit Articles from the seed.js file as it is our Single Source of Truth.

In order for this to be shown in all the other Views we will need to update every single Route to change from the previous sidebar to the new one that fetches records from the DB.

We also only need a single article.\$slug.tsx file that will manage all the article backend:

```

app > routes > article.$slug.tsx > ArticlePage
1  import { LoaderFunctionArgs, json } from "@remix-run/node";
2  import { useLoaderData } from "@remix-run/react";
3  import PageTemplate from "app/components/ui/pagetemplate";
4  import SidebarArticles from "app/components/ui/sidebararticles";
5  import Article from "app/components/ui/article";
6  import prisma from "app/db.server";
7
8  export async function Loader({ params }: LoaderFunctionArgs) {
9    const article = await prisma.article.findUnique({ where: { slug: params.slug } });
10    if (!article) throw new Response("Not Found", { status: 404 });
11    const articles = await prisma.article.findMany({
12      select: { title: true, slug: true },
13      orderBy: { title: "asc" },
14    });
15    return json({ article, articles });
16  }
17
18  export default function ArticlePage() {
19    const { article, articles } = useLoaderData<typeof Loader>();
20    return (
21      <PageTemplate sidebar={<SidebarArticles articles={articles} />}>
22        <Article
23          image={article.image}
24          imageAlt={article.imageAlt}
25          title={article.title}
26          subtitle={article.subtitle}
27        >
28          <div dangerouslySetInnerHTML={{ __html: article.content }} />
29        </Article>
30      </PageTemplate>
31    );
32  }

```

We have successfully made it possible to not need a single route for every new article.

That would be all for features, I would actually have loved to add nested comments in the future but I kind of run late with time... Could probably retake this project in the future and finish it :)

6. Optimisations

Adding FontAwesome

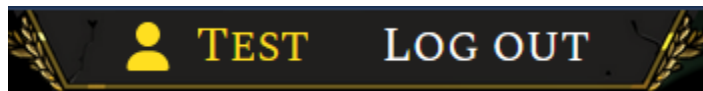
I will be focusing from here on improving the Aesthetic of the webpage by improving the styles and adding some icons/colors. By doing this we can import directly into every template.

```
pnpm add @fortawesome/react-fontawesome  
@fortawesome/free-solid-svg-icons
```

```
4 | import { FontAwesomeIcon } from '@fortawesome/react-fontawesome';  
5 | import { faUser } from '@fortawesome/free-solid-svg-icons';
```

I can't stress enough how convenient this is as I can just change the width (w) or height (h) so easily this way instead of having to write actual CSS for it.

```
<span className="relative z-10 text-yellow-300 pl-2 px-1 flex items-center">  
  <FontAwesomeIcon icon={faUser} className="w-4 h-4 mr-2" />  
  {user.username}  
</span>
```



Minor Adjustments

Mainly just playing a bit with Tailwind v4 to edit some templates and obtain further visual consistency for a smoother UI/UX experience.

Also populated many Articles with actual in-game entries to make it look better for deployment.

Routed all images inside /public/images

Added a better main page to introduce the game with their latest patch notes.

Testing and Deployment

All test scenarios have been run locally and we have followed many flow scenarios such as:

- Registering
- Logging out
- Logging in with Username/Email
- Changing viewport (Responsiveness)
- Creating articles from the seed.js file

Deployment will be made in Render. First, we create a Postgres DB inside Render:

New Postgres [View docs](#)

Name
A unique name for your Postgres instance.
Hades 2

Project Optional
Add this database to a project once it's created.
[Create a new project to add this to?](#)
You don't have any projects in this workspace. Projects allow you to group resources into environments so you can better manage related resources.
[+ Create a project](#)

Database Optional
The Postgres dbname
wiki_pro

User Optional
randomly generated unless specified

Region
Your services in the same region can communicate over a private network.
Frankfurt (EU Central)

Connections

Hostname
An internal hostname used by your Render services.
dpg-d16chts9c44c73dj2300-a

Port
5432

Database
wiki_pro

Username
wiki_pro_user

Password
[Redacted]

Internal Database URL
[Redacted]

External Database URL
[Redacted]

PSQL Command
[Redacted]

We then paste the contents of the Internal Database URL in our Environment Variables later on.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

.....

+ Add Environment Variable

 Add from .env

It's time to create a “New Web Service” and then it will be just as simple as connecting our GitHub repository to the Project.

You are deploying a Web Service

You seem to be using **Node**, so we've autofilled some fields accordingly. Make sure the values look right to you!

Source Code

 philipyang89 / hades2 • 9m ago

Name

hades2

A unique name for your web service.

We then define our Build Commands to successfully set up the production environment.

Build Command

Render runs this command to build your app before each deploy.

\$ --frozen-lockfile; pnpm run build ; pnpm prisma migrate deploy ; pnpm prisma generate ; pnpm prisma db seed

 WEB SERVICE

hades2

Node

Free

Upgrade your instance →

 philipyang89 / hades2  main

<https://hades2.onrender.com> 

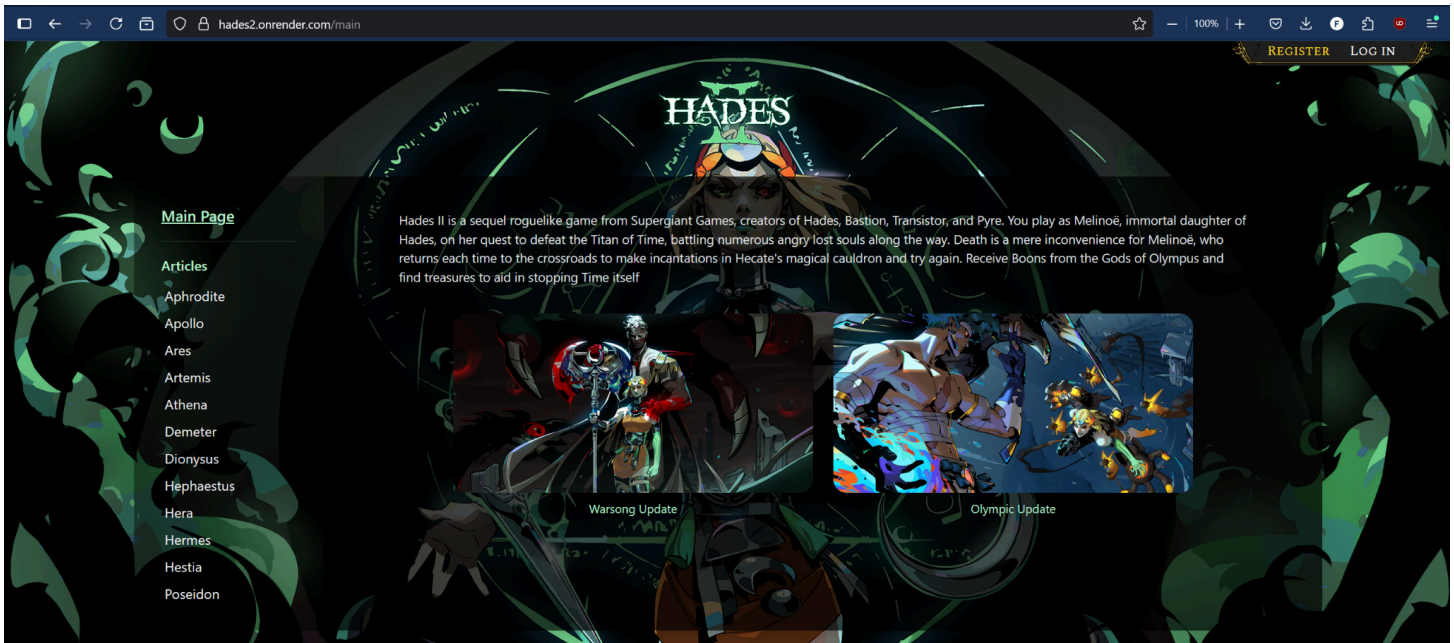
```
All logs  Search
Jun 5 12:46:07 AM  Tip: Interested in query caching in just a few lines of code
Jun 5 12:46:07 AM
Jun 5 12:46:07 AM  |
Jun 5 12:46:07 AM  | Update available 6.8.2 -> 6.9.0
Jun 5 12:46:07 AM  | Run the following to update
Jun 5 12:46:07 AM  |   npm i --save-dev prisma@latest
Jun 5 12:46:07 AM  |   npm i @prisma/client@latest
Jun 5 12:46:07 AM  |
Jun 5 12:46:08 AM  Running seed command `node prisma/seed.js` ...
Jun 5 12:46:08 AM
Jun 5 12:46:08 AM  🌱 The seed command has been executed.
Jun 5 12:46:09 AM  ==> Uploading build...
Jun 5 12:46:24 AM  ==> Deploying...
Jun 5 12:46:21 AM  ==> Uploaded in 8.5s. Compression took 3.6s
Jun 5 12:46:21 AM  ==> Build successful 🎉
```

So we can now successfully enter <https://hades2.onrender.com> and basically verify that all of the requests are being handled correctly in our console log:

```
Jun 5 12:47:45 AM  GET /article/aphrodite.data 200 - - 8.486 ms
Jun 5 12:47:46 AM  GET /article/apollo.data 200 - - 6.005 ms
Jun 5 12:47:54 AM  GET /article/ares.data 200 - - 7.487 ms
Jun 5 12:47:57 AM  GET /article/demeter.data 200 - - 5.633 ms
Jun 5 12:47:57 AM  GET /article/dionysus.data 200 - - 6.561 ms
Jun 5 12:47:58 AM  GET /article/hephaestus.data 200 - - 4.484 ms
```

And that would mark the ending of our project!

Conclusions



This project ended up taking a bit longer than I expected and there are some features such as Nested Comments that I would like to implement looking forward in the future.

I definitely learned a lot this time, I managed to pick up a full-stack framework in React such as Remix.js despite my 0 knowledge on this previously.

I want to thank my work teammates for reaching a hand at times where I needed help, especially on the backend side since that's what I'm worse at.

I also learnt a lot from frontend with Tailwind v4 and it's a shame I ended up not using shadcn/ui components at all despite me initially wanting to. I believe they are more suitable for websites with shopping or accounting features, really. I didn't manage to find one suitable so I ended up doing all components from scratch.

This project has been a good step towards being that Software Engineer I dream of - that will be able to create many types of applications and hopefully contribute and help other people.

References

Hades 2 SteamDB: <https://steamdb.info/app/1145350/info/>

Wikipedia Inspirations: [Terraria](#) , [Hollow Knight](#) , [Stardew Valley](#)

Remix + shadcn/ui Setup: <https://ui.shadcn.com/docs/installation/remix>

Tailwind CSS v4.0 Changelog: <https://tailwindcss.com/blog/tailwindcss-v4>

Tailwind v4 Upgrade: <https://ui.shadcn.com/docs/tailwind-v4>

<https://tailwindcss.com/docs/upgrade-guide>

Prisma Docs: <https://www.prisma.io/docs/getting-started>

<https://www.prisma.io/docs/orm/prisma-schema/overview>

Convert into oklch: <https://oklch.com>

CSS in Remix: [Medium](#)

Radix UI: <https://www.radix-ui.com/>

(Framer) Motion: <https://motion.dev/>

Motion Examples: <https://examples.motion.dev/>

Deployment for Remix comparison: <https://www.jacobparis.com/content/where-to-host-remix>

pnpm/npm/yarn: <https://youtu.be/ZIKDJBrk56k>

Decompressor Tool: <https://github.com/quaerus/depPTH>

Python Pillow: <https://python-pillow.github.io/>

Authentication Dependencies: <https://www.npmjs.com/package/@ganmahmud/remix-auth-jwt>